



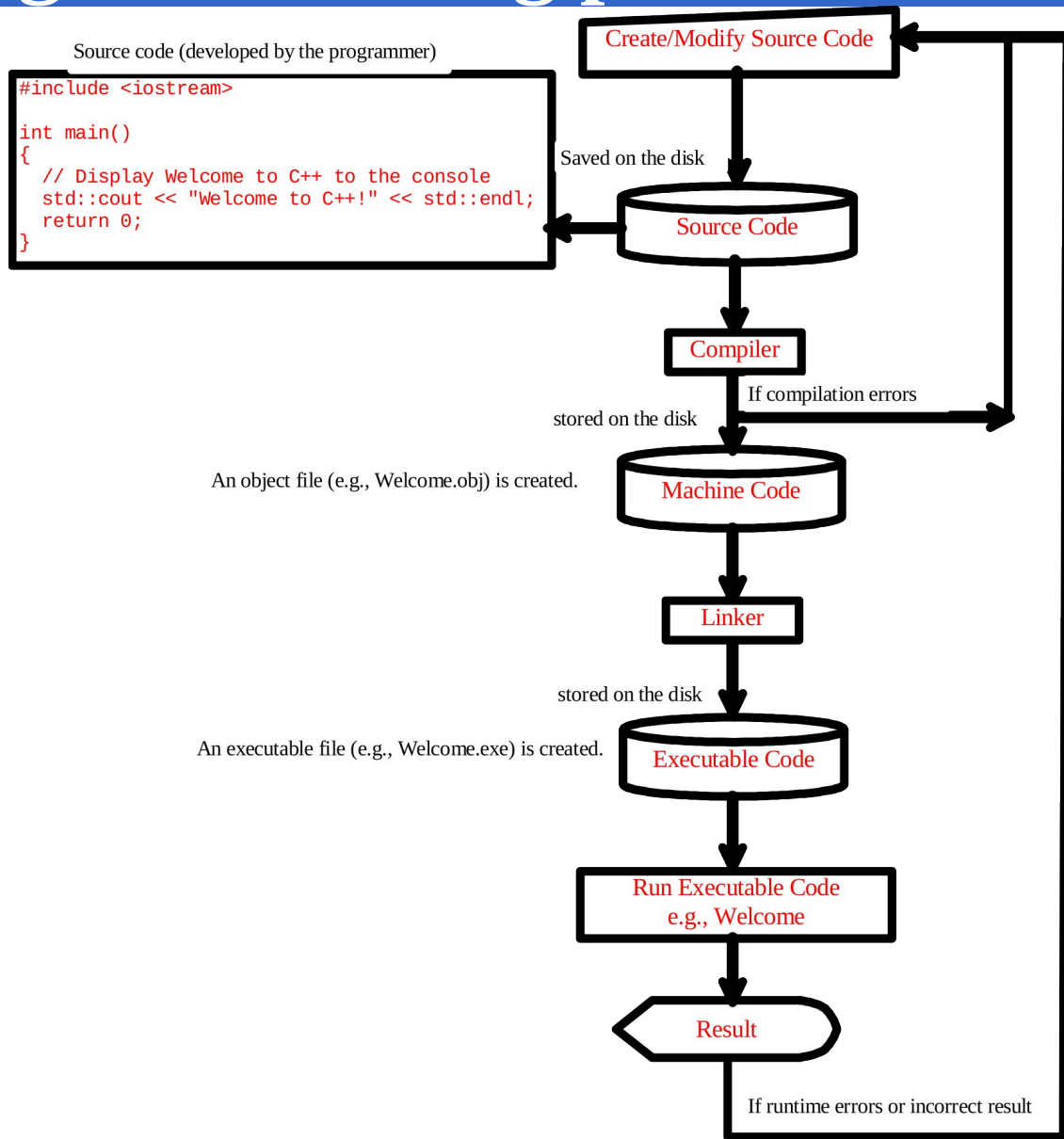
# 程序设计原理

## 循环结构

任课教师：喻 梅  
于瑞国  
王建荣  
李雪威  
赵满坤



# Program running process





# Programming Errors

**Syntax Errors:** Errors that are detected by the compiler are called syntax errors or compile errors. Syntax errors result from errors in code construction, such as mistyping a keyword, omitting necessary punctuation, or using an opening brace without a corresponding closing brace.

```
#include <iostream>
using namespace std

int main()
{
    cout << "Programming is fun << endl;

    return 0;
}
```



# Programming Errors

```
1  #include <iostream>
2  using namespace std
3
4  int main()
5  {
6      cout << "Programming is fun << endl;
7
8      return 0;
9  }
10
11
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, \*\*/\*.ts, !\*\*/nod...     

✓  main.cpp calculator\ch6 6

-  expected ';' before 'int' gcc [Ln 2, Col 20]
-  expected a ';' C/C++(65) [Ln 4, Col 1]
-  missing terminating " character gcc [Ln 6, Col 11]
-  missing closing quote C/C++(8) [Ln 6, Col 11]
-  expected primary-expression before 'return' gcc [Ln 8, Col 3]
-  missing terminating " character gcc [Ln 6, Col 11]



# Programming Errors

**Runtime Errors:** cause a program to terminate abnormally. They occur while an application is running if the environment detects an operation that is impossible to carry out. Such as input errors, division by zero.

```
#include <iostream>
using namespace std;

int main()
{
    int i = 4;
    int j = 0;
    cout << i / j << endl;
    return 0;
}
```



# Programming Errors

**Logic Errors:** when a program does not perform the way it was intended.

```
#include <iostream>
using namespace std;
```

```
int main()
{
    cout << "Celsius 35 is Fahrenheit degree " << endl;
    cout << (9 / 5) * 35 + 32 << endl;

    return 0;
}
```

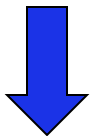


# Common Errors

## Common Error 1: Missing Braces

```
#include <iostream>
using namespace std;
```

```
int main()
{
    cout << "Welcome to C++!" << endl;
    cout << "Welcome to C++.NET!" << endl;
    cout << "Welcome to C++ Compiler!" << endl;
    return 0;
```



**Missing Brace**

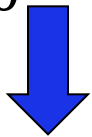


# Common Errors

## Common Error 2: Missing a semicolon

```
#include <iostream>
using namespace std;
```

```
int main()
{
    cout << "Welcome to C++!" << endl;
    cout << "Welcome to C++.NET!" << endl;
    cout << "Welcome to C++ Compiler!" << endl;
    return 0
}
```



**Missing a semicolon**



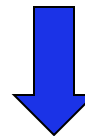


# Common Errors

## Common Error 3: Missing a quotation mark

```
#include <iostream>  
using namespace std;
```

```
int main()  
{  
    cout << "Welcome to C++!" << endl;  
    cout << "Welcome to C++.NET!" << endl;  
    cout << "Welcome to C++ Compiler!<< endl;  
    return 0;  
}
```



**Missing a quotation mark**



# Common Errors

## Common Error 4: Spelling Error

```
#include <iostream>
using namespace std;
```

```
int Main()  Spelling Error
{
    cout << "Welcome to C++!" << endl;
    cout << "Welcome to C++.NET!" << endl;
    cout << "Welcome to C++ Compiler! " << endl;
    return 0;
}
```



# Exercise

## Swapping Two Numbers

Write a program which reads two integers  $x$  and  $y$ , and prints them in ascending order.

### Input

The input consists of multiple datasets.  
Each dataset consists of two integers  $x$  and  $y$  separated by a single space.

The input ends with two 0 (when both  $x$  and  $y$  are zero). Your program should not process for these terminal symbols.

### Output

For each dataset, print  $x$  and  $y$  in ascending order in a line. Put a single space between  $x$  and  $y$ .

### Constraints

$0 \leq x, y \leq 10000$ , the number of datasets  $\leq 3000$

### Sample Input

3 2

2 2

5 3

0 0

### Sample Output

2 3

2 2

3 5



```
#include <iostream>
using namespace std;

int main() {
    int x, y;

    cin >> x >> y;

    while (x != 0 && y != 0) {
        if (x > y) {
            int temp = x;
            x = y;
            y = temp;
        }
        cout << x << " " << y << endl;
    }

    return 0;
}
```



```
#include <iostream>
using namespace std;

int main() {
    int x, y;

    cin >> x >> y;

    while (x != 0 || y != 0) {
        if (x > y) {
            int temp = x;
            x = y;
            y = temp;
        }
        cout << x << " " << y << endl;
    }

    return 0;
}
```



```
1 ▾ #include <iostream>
2   using namespace std;
3
4 ▾ int main() {
5     int x, y;
6
7     cin >> x >> y;
8
9     while (x != 0 || y != 0) {
10 ▾     if (x > y) {
11         int temp = x;
12         x = y;
13         y = temp;
14     }
15     cout << x << " " << y << endl;
16     cin >> x >> y;
17 }
18
19     return 0;
20 }
```



# Exercise

## Print a Rectangle

Draw a rectangle which has a height of  $H$  cm and a width of  $W$  cm. Draw a 1-cm square by single '#'.

### Input

The input consists of multiple datasets. Each dataset consists of two integers  $H$  and  $W$  separated by a single space.

The input ends with two 0 (when both  $H$  and  $W$  are zero).

### Output

For each dataset, print the rectangle made of  $H \times W$  '#'.

Print a blank line after each dataset.

## Constraints

$$1 \leq H \leq 300$$

$$1 \leq W \leq 300$$

## Sample Input

3 4  
5 6  
2 2  
0 0

## Sample Output

####  
####  
####  
  
#####  
#####  
#####  
#####  
#####  
  
##  
##



# Exercise

## Print a Frame

Draw a frame which has a height of  $H$  cm and a width of  $W$  cm. For example, the following figure shows a frame which has a height of 6 cm and a width of 10 cm.

```
#####  
#.....#  
#.....#  
#.....#  
#.....#  
#####
```

## Input

The input consists of multiple datasets. Each dataset consists of two integers  $H$  and  $W$  separated by a single space.

The input ends with two 0 (when both  $H$  and  $W$  are zero).

## Output

For each dataset, print the frame made of '#' and '.'.

Print a blank line after each dataset.





# Exercise

## Constraints

$$3 \leq H \leq 300$$

$$3 \leq W \leq 300$$

## Sample Input

3 4

5 6

3 3

0 0

## Sample Output

####

#..#

####

#####

#...#

#...#

#...#

#####

###

#.#

###



# Exercise

## Print a Chessboard

Draw a chessboard which has a height of  $H$  cm and a width of  $W$  cm. For example, the following figure shows a chessboard which has a height of 6 cm and a width of 10 cm.

```
#.#.#.#.#.  
.#.#.#.#.  
#.#.#.#.#.  
.#.#.#.#.  
#.#.#.#.#.  
.#.#.#.#.
```

Note that the top left corner should be drawn by '#'.

## Input

The input consists of multiple datasets. Each dataset consists of two integers  $H$  and  $W$  separated by a single space.

The input ends with two 0 (when both  $H$  and  $W$  are zero).

## Output

For each dataset, print the chessboard made of '#' and '.'.

Print a blank line after each dataset.



# Exercise

## Constraints

$$1 \leq H \leq 300$$

$$1 \leq W \leq 300$$

## Sample Input

3 4

5 6

3 3

2 2

1 1

0 0

## Sample Output

##.

.##.

##.

###.

####.

###.

####.

###.

##

.#.

##

#.

.#

#



# Exercise

## Structured Programming

In programming languages like C/C++, a goto statement provides an unconditional jump from the "goto" to a labeled statement. For example, a statement "goto CHECK\_NUM;" is executed, control of the program jumps to CHECK\_NUM. Using these constructs, you can implement, for example, loops.

Note that use of goto statement is highly discouraged, because it is difficult to trace the control flow of a program which includes goto.

Write a program which does precisely the same thing as the following program (this example is written in C++). Let's try to write the program without goto statements.

```
void call(int n){
    int i = 1;
CHECK_NUM:
    int x = i;
    if ( x % 3 == 0 ){
        cout << " " << i;
        goto END_CHECK_NUM;
    }
INCLUDE3:
    if ( x % 10 == 3 ){
        cout << " " << i;
        goto END_CHECK_NUM;
    }
    x /= 10;
    if ( x ) goto INCLUDE3;
END_CHECK_NUM:
    if ( ++i <= n ) goto CHECK_NUM;

    cout << endl;
}
```



# Exercise

## Input

An integer  $n$  is given in a line.

## Output

Print the output result of the above program for given integer  $n$ .

## Constraints

$$3 \leq n \leq 10000$$

## Sample Input

30

## Sample Output

3 6 9 12 13 15 18 21 23 24 27 30

Put a single space character before each element.